The CLAUDE.md File

One of the most useful features in Claude Code. The CLAUDE.md file gives Claude persistent memory about your project - an onboarding script it reads at the start of every session.

Persistent context · /init · Structure & best practices

IN THIS GUIDE

Give Claude persistent memory of your project so you stop re-explaining it every session. We'll cover:

- What a `CLAUDE.md` is and the problem it solves
- What to put in it - and where it lives (the **memory hierarchy**)
- Bootstrapping one fast with `/init`
- Structuring it well - a **map**, your **tools**, and standard **workflows**
- Techniques that pair well, and how to **keep it focused and safe**

ON THIS PAGE

01 The Problem It Solves

When you open Claude Code without a CLAUDE.md, it starts fresh every time. It re-explores your codebase, figures out which dependencies are needed, and works out what's already implemented - sometimes making assumptions that are hard to steer.

The problem compounds as a codebase grows. Complex module relationships, domain-specific patterns, and team conventions don't surface easily, so you end up re-explaining the same architecture, testing requirements, and code-style preferences at the start of every conversation.

CLAUDE.md solves this. It's a Markdown file at the root of your project that Claude reads automatically every session. Think of it as an **onboarding script** for your codebase - its contents are appended to your prompt and become part of Claude's system prompt, so every conversation starts with that context already loaded.

02 What Goes in a CLAUDE.md

A CLAUDE.md can document common commands, core utilities, code-style guidelines, testing instructions, repository conventions, dev-environment setup, and project-specific warnings. There's no required format - keep it concise and human-readable, like documentation both people and Claude can scan quickly. Each addition should solve a real problem you've hit, not a theoretical one.

Here's what a typical CLAUDE.md looks like:

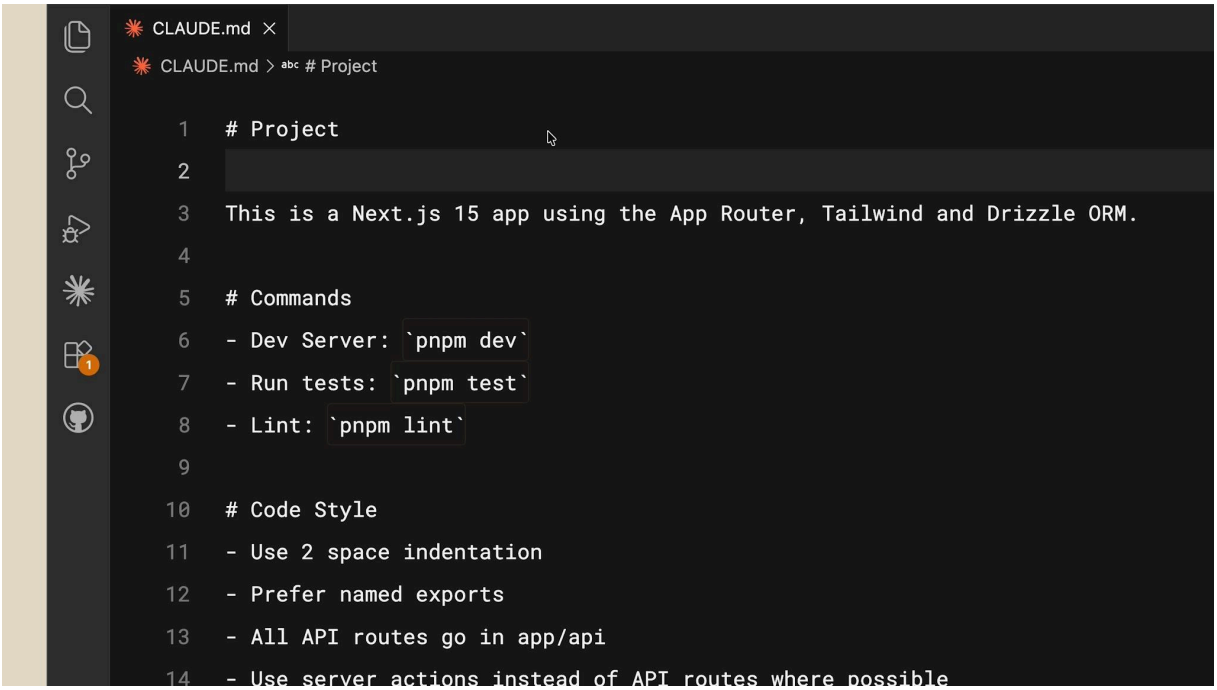
CLAUDE.MD

```
# Project
This is a Next.js 15 app using the App Router, Tailwind, and
Drizzle ORM.

# Commands
- Dev server: `npm dev`
- Run tests:  `npm test`
- Lint:       `npm lint`

# Code Style
- Use 2-space indentation
- Prefer named exports
```

```
- All API routes go in app/api/
- Use server actions instead of API routes where possible
```



A typical CLAUDE.md - project summary, commands, and code-style conventions.

It's straightforward. Now if you ask Claude to create a React component, it already knows to use Tailwind for styling and to follow your conventions - no reminders needed. The file becomes part of Claude's system prompt, so that context loads on every conversation.

03 Where It Lives: The Memory Hierarchy

There's a hierarchy of memory files depending on who they're for. Commit the project-level file to version control so your whole team benefits from it.

Scope	Location	Shared with
Project	Repository root (`CLAUDE.md`)	Your whole team - commit it to version control.
Monorepo	A parent directory above the package	Everything nested under that path.

Scope	Location	Shared with
Personal	Your home / configuration folder	Just you - applies across all your projects.

Put your personal preferences in the user-level file and team conventions in the project-level one. For monorepos, a CLAUDE.md in a parent directory applies to everything beneath it.

04 Getting Started with /init

Creating a CLAUDE.md from scratch can feel daunting, especially in an unfamiliar codebase. The `/init` command automates it - it analyzes your project and generates a starter configuration. Run it in any Claude Code session:

TERMINAL

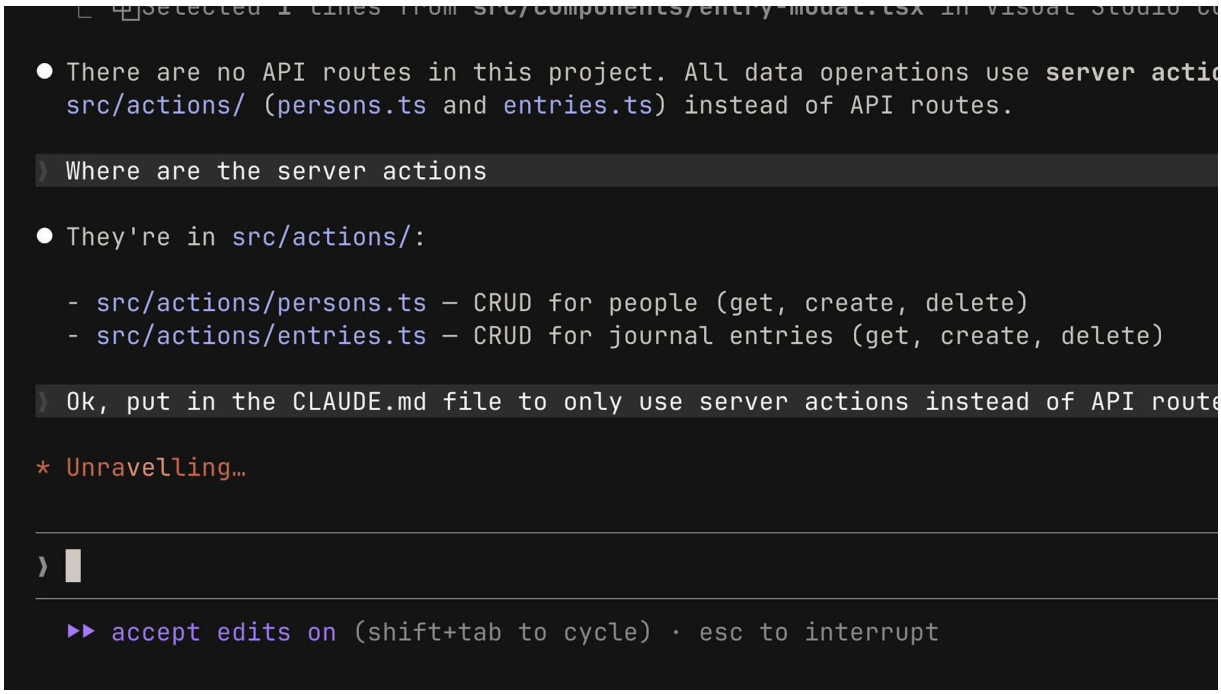
```
cd your-project
claude
/init
```

Claude reads package files, existing documentation, configuration, and code structure, then generates a CLAUDE.md tailored to your project - typically build commands, test instructions, key directories, and the conventions it detected. It also works on projects that *already* have a CLAUDE.md, reviewing the current file and suggesting improvements.

Treat the result as a **starting point, not a finished product**. After running `/init`:

- Review the generated content for accuracy.
- Add workflow instructions Claude couldn't infer - branch-naming conventions, deployment processes, code-review requirements.
- Remove generic guidance that doesn't apply to your project.
- Commit the file to version control so your team benefits.

Grow it as you go. The real value comes from iterating. As you work, use the `#` key to add instructions you find yourself repeating - or just ask Claude to save a recurring correction straight to your CLAUDE.md.



Ask Claude to save a recurring correction straight into your CLAUDE.md - next session it already knows.

05 How to Structure Your CLAUDE.md

A few patterns give your file the most leverage: a map of the codebase, your custom tools, and standard workflows.

Give Claude a map

Explaining your architecture, key libraries, and styles for every task gets tedious. Add a project summary and a high-level directory tree so Claude is instantly oriented when navigating the codebase:

PROJECT MAP

```
main.py
├── logs/
│   └── application.log
├── modules/
│   ├── cli.py
│   ├── logging_utils.py
```

```
|   ├── media_handler.py
|   └── player.py
```

Include your main dependencies, architectural patterns, and any non-standard choices - domain-driven design, microservices, specific frameworks. Claude uses this map to decide where to find code and where to make changes.

Connect Claude to your tools

Claude inherits your environment but needs guidance on which custom tools and scripts to use. Document them with usage examples - names, basic usage, when to invoke them, and a `--help` flag if one exists. Claude is also an MCP (Model Context Protocol) client; configure servers through project settings, global config, or a checked-in `.mcp.json` (the `--mcp-debug` flag helps troubleshoot). For example:

CLAUDE.MD

```
### Slack MCP
- Posts to #dev-notifications channel only
- Use for deployment notifications and build failures
- Do not use for individual PR updates (those go through
  GitHub webhooks)
- Rate limited to 10 messages per hour
```

Define standard workflows

Jumping straight into code changes creates rework - Claude might miss requirements, pick the wrong approach, or break existing behavior. Define workflows it should follow. A solid default answers four questions before changing code:

- Is this a question about the current state that needs investigation first?
- Does this need a detailed plan before implementation?
- What additional information is missing?
- How will effectiveness be tested?

Name specific workflows: **explore** → **plan** → **code** → **commit** for features, test-driven development for algorithmic work, or visual iteration for UI. Document your testing requirements,

commit-message format, and any approval steps so Claude matches your team's actual process instead of guessing.

06 Techniques That Pair Well

Three habits beyond the file itself keep Claude sharp.

Keep context fresh with /clear

Over a session, old file contents and command outputs pile up and drown out the signal. Run `/clear` between distinct tasks to reset the context window - it keeps your CLAUDE.md loaded while clearing the accumulated history. When you finish debugging auth and switch to a new endpoint, clear it; the auth details no longer matter.

Use subagents for distinct phases

Long conversations carry bias. When a phase needs fresh eyes - say a security review of code you just wrote - tell Claude to use a subagent. It runs in an isolated context, so the debugging details don't color the review. Context separation keeps both analyses sharp.

Create custom commands

Store repeated prompts as Markdown in your `.claude/commands/` directory. A file named `performance-optimization.md` becomes `/performance-optimization` in any session; commands take arguments through `$ARGUMENTS` or numbered placeholders like `$1`. Here's an abbreviated example:

PERFORMANCE-OPTIMIZATION.MD

```
# Performance Optimization

Analyze the provided code for performance bottlenecks across:
- Database & data access - N+1 queries, missing indexes,
  pagination
- Algorithm efficiency - time complexity, redundant work
- Memory, async/concurrency, network & I/O, and caching
```

```
For each issue, give: location, severity, and a concrete fix.

Code to review:
$ARGUMENTS
```

You don't need to write these by hand - ask Claude to create one:

› Create a custom slash command called /performance-optimization that analyzes code for database query issues, algorithm efficiency, memory management, and caching opportunities.

Claude writes the Markdown file to `.claude/commands/` and the command is available immediately.

07 Keep It Focused & Safe

- **Start lean.** It's tempting to write a comprehensive file up front - resist. One good move: start a project *without* a CLAUDE.md so you can see where you constantly course-correct, then capture only those rules. It keeps the file focused on what actually matters.
- **Keep it concise.** CLAUDE.md is added to context every time, so from a context- and prompt-engineering standpoint, every line costs a lot. Break large content into separate Markdown files and reference them with the `@` symbol (e.g. `@README.md`) to keep the main file tight.
- **Don't include secrets.** No API keys, credentials, connection strings, or detailed vulnerability information - especially if you commit to version control. Treat CLAUDE.md as documentation that could be shared publicly, since it becomes part of Claude's system prompt.
- **Iterate.** Treat customization as an ongoing practice, not a one-time setup. Projects change, teams learn better patterns, and new tools arrive - a well-maintained CLAUDE.md evolves with your codebase.

- Recap

KEY TAKEAWAYS

- CLAUDE.md gives Claude persistent, project-specific context - loaded into every conversation.
- Start with your stack, commands, and preferences; bootstrap with `/init`.
- Structure it with a **map**, your **tools**, and standard **workflows**.
- Keep it concise, reference other docs with `@`, and never commit secrets.
- Iterate - the best CLAUDE.md files capture the real friction in your workflow.

A few minutes documenting your project saves re-explaining it every session.

Hands-On Workshop: Create a CLAUDE.md for Your Project

Give Claude Code Persistent Project Memory

project root · auto-loaded · conventions

IN THIS DEMO

- Create a `CLAUDE.md` that describes our project, commands, and code style.
- Start a session and let Claude load that context on its own.
- Ask Claude to write code and see it follow our conventions with no reminders.

ON THIS PAGE

01 What CLAUDE.md Does

`CLAUDE.md` is a Markdown file at the root of your project. Claude Code reads it automatically at the start of every session and adds it to its context - so each conversation begins already

knowing your stack, commands, and conventions. It is an onboarding script for your codebase: write the project's rules once, and you stop re-explaining them every time.

02 Prerequisites

- Claude Code installed and running (`claude` is available in your terminal).
- A working directory for the exercise.

```
mkdir -p ~/claude-md-lab && cd ~/claude-md-lab
```

Project Folder : Link

03 Create the CLAUDE.md

You can generate a starter automatically with `/init`, which inspects your project and drafts a file for you. For this demo we will write a short one directly so the conventions are explicit.

Create `CLAUDE.md` in the project root:

CLAUDE.md

```
# Project
Camera-surveillance platform. TypeScript backend that ingests RTSP/ONVIF
camera streams and exposes a device API.

# Map
- src/auth/         device authentication (ONVIF digest validation)
- src/integrations/ per-camera integration modules
- src/stream/       stream ingestion and transport
- src/api/          public device API routes

# Commands
- Dev server: `npm run dev`
- Run tests:  `npm test`
- Lint:       `npm run lint`

# Code Style
- 2-space indentation, named exports only.
- All camera/device endpoints must use TLS (rtsps:// or https://),
```

```
  never plain rtsp:// or http://.
- Never hardcode credentials or device tokens; read them from config.
- Validate any host or query value from a caller before using it in a URL.

# Workflow
For new integrations: explore the existing module in src/integrations/
first, then plan, then implement, then add a test.
```

Notice this captures real friction for this codebase - TLS-only endpoints, no hardcoded credentials, input validation - not generic advice.

04 Start a Session

Open Claude Code in the project so it loads the file:

```
claude
```

You can confirm it was picked up by asking:

```
What do you know about this project?
```

Claude should summarize the stack, directory map, and conventions straight from your `CLAUDE.md` - without you pasting anything.

05 Watch It Follow Your Conventions

Now ask for a task without restating any rules:

```
Add a new integration module for an Axis camera in src/integrations/
```

Claude applies what the file already told it: it uses a TLS endpoint rather than plain `http://`, reads credentials from config instead of hardcoding them, validates caller input, uses named exports and 2-space indentation, and follows the explore-then-plan-then-implement workflow. None of that was in your request - it came from `CLAUDE.md`.

06 Capture a Recurring Correction (Optional)

When you find yourself correcting Claude on the same thing, save it to the file so it sticks for next time. Either type a line starting with `#` to append it, or just ask:

```
Add a note to CLAUDE.md: log stream URLs without their query string,
since the query can contain a token.
```

Claude updates `CLAUDE.md`, and from the next session that rule is part of its context automatically.

07 Summary

- `CLAUDE.md` lives at the project root and loads into every session automatically.
- Fill it with a project map, your commands, your code style, and standard workflows.
- Bootstrap quickly with `/init`, then refine - capture only the rules that reflect real friction.
- Keep it concise; every line costs context. Reference larger docs with `@filename` instead of pasting them.
- Never include secrets - treat it as documentation that could be shared, since it becomes part of Claude's context.

08 Troubleshooting

Symptom	Resolution
Claude ignores the conventions	Confirm the file is named exactly `CLAUDE.md` and sits in the directory where you launched `claude`.
File is getting long and unfocused	Move detailed sections into separate Markdown files and reference them with `@`.
A rule keeps being missed	Make it more specific and concrete; vague guidance is easy to overlook.

09 Next Steps

- Commit `CLAUDE.md` so the whole team shares the same conventions.

- Add a personal user-level memory file for preferences that apply across all your projects.
- Pair it with subagents for isolated phases (such as a security review) and `/clear` to reset context between unrelated tasks while keeping `CLAUDE.md` loaded.

> **Write the project's rules once - and Claude starts every session already knowing them.**